Ecriture de code et tests unitaires

Boitelle Clémence

Introduction

Dans ce document, nous analysons et corrigeons le fichier Exemple.java.
L'objectif est de :

- Identifier les problèmes présents dans le code initial
- Comprendre pourquoi ces problèmes sont bloquants ou mauvais pour la qualité du code
- Proposer des corrections adaptées
- Vérifier le code corrigé à l'aide des outils Checkstyle, SpotBugs et PMD

Ce document permet de comprendre le travail réalisé sans consulter directement le code source.

Méthodologie

Pour chaque problème détecté :

- Problème : description de l'anomalie dans le code
- Pourquoi c'est un problème : explication technique ou pratique
- Réécriture proposée : correction du code
- Rapports après correction : état des outils après modification

Exercice 1 : Analyse et correction du code

➢ Erreur: cannot assign a value to final variable v

```
private final int v;
public final void setV(int v){this.v =v;}
```

Pourquoi c'est un problème

La variable ***v*** est déclarée ***final***, ce qui signifie qu'elle **ne peut pas être modifiée** après son initialisation. La méthode ***setV*** tente de modifier ***v***, ce qui provoque une erreur de compilation. Une variable ***final*** doit être initialisée **exactement une fois**, généralement dans le constructeur ou à sa déclaration.

Réécriture proposée :

```
private int v;

public final void setV(int v){

   this.v =v;}
```

➢ Erreur : missing return statement

```
public final String getT() {

   if ( v > 0) return t;

}
```

Pourquoi c'est un problème

- La méthode est déclarée pour retourner un ***String***.
- Si ***v <= 0***, aucune valeur n'est retournée, ce qui provoque une **erreur de compilation**.
- Toutes les branches d'une méthode non ***void*** doivent retourner une valeur.

Réécriture proposée :

```
public final String getT() {

   if ( v > 0) return t;

   return ""; // retourne une valeur par défaut

}
```

➢ **Erreur : variable t might not have been initialized**

```
private final String t;
public Exemple(String t) {

   t = t;

}
```

Pourquoi c'est un problème

- Le champ t est déclaré ***final***, donc il **doit être initialisé exactement une fois**.
- L'affectation ***t = t;*** **ne fait que affecter le paramètre à lui-même**, le champ **t** reste donc non initialisé.
- Java bloque la compilation si un champ final n'est pas initialisé.

Réécriture proposée:

```
public Exemple(String t) {

   this.t = t;

}
```

Compilation du projet

Après les corrections apportées au code, le projet compile sans erreur.

[INFO] BUILD SUCCESS

Cela indique que la compilation s'est déroulée correctement et que le code peut être exécuté.

Cependant, **cela ne signifie pas qu'il est totalement exempt d'erreurs** : il peut encore contenir des **problèmes de style, de conception ou de logique** que la compilation ne détecte pas.

C'est pourquoi il est nécessaire d'analyser le code avec des outils tels que **Checkstyle**, **SpotBugs** et **PMD**, afin d'identifier d'éventuelles mauvaises pratiques ou anomalies.

Rapport Checkstyle

➢ Qu’est-ce que Checkstyle ?

Checkstyle est un outil d’analyse statique qui vérifie que le code Java respecte un ensemble de règles de style et de bonnes pratiques.
Il ne détecte pas directement des erreurs d’exécution, mais il aide à maintenir un code :

- lisible
- cohérent
- facile à maintenir
- conforme aux conventions de codage (par exemple Sun/Oracle).

Son rôle est donc d’imposer une discipline de codage, afin d’éviter des problèmes de lisibilité, de documentation et de qualité logicielle à long terme.

➢ Résultats globaux

Les résultats de Checkstyle montrent **15 erreurs** dans le fichier `Exemple.java`.
Ces erreurs concernent principalement :

- la documentation Javadoc
- l’utilisation de paramètres masquant des attributs
- le manque de cohérence sur les accolades et espaces
- des règles de bonne pratique comme `final` pour les paramètres.

Captures d’écran du rapport Checkstyle :

Résultats Checkstyle

Le document suivant contient les résultats de Checkstyle 9.3 avec les règles sun_checks.xml.

Résumé

Fichiers	Infos	Avertissements	Erreurs
2	0	0	15

Fichiers

Fichier	I	A	E
Exemple.java	0	0	15

Règles

Catégorie	Règle	Violations	Sévérité
blocks	NeedBraces	1	Erreur
coding	HiddenField	2	Erreur
javadoc	JavadocVariable	2	Erreur
	MissingJavadocMethod	2	Erreur
misc	FinalParameters	2	Erreur
	NewlineAtEndOfFile	1	Erreur
whitespace	ParenPad	1	Erreur
	WhitespaceAfter	1	Erreur
	WhitespaceAround	3	Erreur

Détails

Exemple.java

Sévérité	Catégorie	Règle	Message	Ligne
Erreur	misc	NewlineAtEndOfFile	Il manque un caractère NewLine à la fin du fichier.	1
Erreur	javadoc	JavadocVariable	Commentaire Javadoc manquant.	5
Erreur	javadoc	JavadocVariable	Commentaire Javadoc manquant.	6
Erreur	javadoc	MissingJavadocMethod	Commentaire Javadoc manquant.	7
Erreur	misc	FinalParameters	Le paramètre t devrait être final.	7
Erreur	coding	HiddenField	't' masque un attribut.	7
Erreur	javadoc	MissingJavadocMethod	Commentaire Javadoc manquant.	16
Erreur	misc	FinalParameters	Le paramètre v devrait être final.	16
Erreur	coding	HiddenField	'v' masque un attribut.	16
Erreur	whitespace	WhitespaceAround	Il manque une espace avant '{'.	16
Erreur	whitespace	WhitespaceAround	Il manque une espace après '='.	17
Erreur	whitespace	WhitespaceAfter	Il manque une espace après ';'.	17
Erreur	whitespace	WhitespaceAround	Il manque une espace avant '}'.	17
Erreur	blocks	NeedBraces	L'instruction 'if' devrait utiliser des accolades ('{' et '}').	22
Erreur	whitespace	ParenPad	Il y a une espace de trop après '('.	22

➢ Erreur : NewlineAtEndOfFile

Message Checkstyle : *« Il manque un caractère NewLine à la fin du fichier »*

Pourquoi c'est un problème

Selon les conventions Unix/Linux, chaque fichier texte doit se terminer par une **ligne vide** (caractère ***\n***).

- Cela permet une meilleure compatibilité entre systèmes, évite des conflits dans Git lors de la concaténation de fichiers, et garantit une lisibilité homogène.
- Sans ce retour à la ligne, certains outils peuvent considérer que deux fichiers fusionnés se trouvent sur la même ligne.

Réécriture proposée :

Ajout simplement d'une **ligne vide** à la fin du fichier ***Exemple.java*** .

➢ Erreur : JavadocVariable (ligne 5 et 6)

Message Checkstyle : *« Commentaire Javadoc manquant »*

Pourquoi c'est un problème ?

Les attributs de classe doivent être documentés avec un commentaire **Javadoc**. Sans ce dernier, un futur développeur ne pourra savoir à quoi sert la variable. Nuisant ainsi à la génération automatique de documentation et à la lisibilité du code.

Code problématique :

```
private final String t;
private int v;
```

Réécriture proposée :

```
/**
* Chaîne de caractères stockée dans l'attribut t.
*/
private final String t;
/**
* Valeur entière stockée dans l'attribut v.
*/
private int v;
```

➢ Erreur : MissingJavadocMethod

Message Checkstyle : « Commentaire Javadoc manquant »

Pourquoi c'est un problème ?

Les méthodes publiques doivent être documentées avec des commentaires Javadoc.
Sans Javadoc, on ne sait pas clairement à quoi sert la méthode, ce qu'elle retourne (***return***) , ou ce qu'elle attend en paramètre (***@param***).
C'est indispensable pour générer automatiquement une documentation claire et faciliter la maintenance du code.

Code problématique :

```
public Exemple(String t) {
    this.t = t;
}
```

Réécriture proposée :

```
/**
* Constructeur de la classe Exemple.
* @param t la chaîne de caractères à stocker dans l'attribut t
*/
public Exemple(String t) {
    this.t = t;
}
```

➢ Erreur : FinalParameters

Message Checkstyle: « *Le paramètre t devrait être final* »

Pourquoi c'est un problème ?

Un paramètre non ***final*** peut être réaffecté par erreur à l'intérieur du constructeur ou d'une méthode. Cela peut provoquer des comportements inattendus. Le mot-clé ***final*** garantit que le paramètre ne sera pas modifié et améliore la robustesse du code.

Code problématique :

```
public Exemple(String t) {
    this.t = t;
}
```

Réécriture proposée :

```
public Exemple(final String t) {
    this.t = t;
}
```

➢ Erreur : HiddenField

Message Checkstyle: « *'t' masque un attribut.*»

Pourquoi c'est un problème ?

Le paramètre du constructeur a le même nom que l'attribut de la classe (***t***).
Même si on écrit ***this.t = t;***, Checkstyle considère que c'est une mauvaise pratique, car cela peut entraîner de la confusion et diminuer la lisibilité du code.

Quand on écrit :

```
public Exemple(final String t) {
    this.t = t;
}
```

Java comprend que le paramètre s'affecte à lui-même. Résultat : l'attribut t de la classe reste non initialisé (il vaut ***null***). C'est une source de bug car l'objet sera créé avec un champ non défini.

Réécriture proposée :

```
/**
* Constructeur de la classe Exemple.
* @param valeurT la chaîne de caractères à stocker dans l'attribut t
*/
public Exemple(final String valeurT) {
    this.t = valeurT;
}
```

➢ Erreur : MissingJavadocMethod

Message Checkstyle: *« Commentaire Javadoc manquant »*

Pourquoi c'est un problème ?

Chaque méthode publique doit être documentée avec une Javadoc.Sans cd dernier, on ignore à quoi sert la méthode ni ce que signifie son paramètre. Cela rend le code moins clair et empêche la génération d'une documentation propre.

Code problématique :

```
public final void setV(int v){
   this.v =v;}
```

Réécriture proposée :

```
/**
* Modifie la valeur de l'attribut v.
* @param v la nouvelle valeur entière à stocker
*/
public final void setV(int v){
   this.v =v;
}
```

➢ Erreur : FinalParameters

Message Checkstyle : *« Le paramètre v devrait être final.»*

Pourquoi c'est un problème ?

- Le paramètre ***v*** de la méthode ***setV*** peut être réaffecté accidentellement à l'intérieur de la méthode.
- Le déclarer ***final*** empêche cette réaffectation, ce qui rend le code plus robuste et plus clair.

Code problématique :

```
public final void setV(int v){
   this.v =v;
}
```

Réécriture proposée :

```
public final void setV(final int v){
    this.v =v;
}
```

➢ Erreur : HiddenField

Message Checkstyle: *« 'v' masque un attribut.»*

Pourquoi c'est un problème ?

Le paramètre de la méthode s'appelle v, tout comme l'attribut de la classe. Donc, dans le corps de la méthode, ***v*** désigne le paramètre et masque l'attribut. Ce qui nous oblige à utiliser ***this.v*** pour lever l'ambiguïté. Pour Checkstyle, c'est une mauvaise pratique car ça nuit à la lisibilité, il est préférable que le paramètre ait un nom différent.

Code problématique :

```
public final void setV(final int v){
    this.v =v;
}
```

Réécriture proposée :

```
/**
* Modifie la valeur de l'attribut v.
* @param valeurV la nouvelle valeur entière à stocker
*/
public final void setV(final int valeurV){
    this.v =valeurV;
}
```

➢ Erreur : WhitespaceAround

Message Checkstyle: : *« Il manque un espace avant '{'.»*

Pourquoi c'est un problème ?

Checkstyle impose une convention de mise en forme pour améliorer la lisibilité. Pour chaque mot-clé (***if***, ***for***, ***while***, ***method signature***) un espace doit être ajouté avant l'accolade ouvrante. Sans cet espace, le code est plus compact et moins agréable à lire.

Code problématique :

```
public final void setV(final int valeurV){
    this.v =valeurV;
}
```

Réécriture proposée :

```
public final void setV(final int valeurV) {
    this.v =valeurV;
}
```

➢ Erreur : WhitespaceAround

Message Checkstyle: « *Il manque un espace après '='* »

Pourquoi c'est un problème ?

Checkstyle impose qu'il y ait toujours un espace autour des opérateurs ***(=, +, -, etc.***). Sans espace, le code devient plus difficile à lire, surtout quand les expressions sont complexes. L'ajout d'un espace améliore la lisibilité et respecte les conventions de style Java.

Code problématique :

```
public final void setV(final int valeurV) {
    this.v =valeurV;
}
```

Réécriture proposée :

```
public final void setV(final int valeurV) {
    this.v = valeurV;
}
```

➢ Erreur : NeedBraces

Message Checkstyle: *« L'instruction 'if' devrait utiliser des accolades ('{' et '}') »*

Pourquoi c'est un problème

En Java, même si une seule instruction suit un ***if***, on peut écrire :

```
if (v > 0) return t;
```

Mais Checkstyle impose l'utilisation d'accolades ***{ }*** pour éviter des bugs potentiels. Par exemple, si quelqu'un rajoute une ligne après le ***if***, sans accolades.

Code problématique :

```
public final String getT() {
   if (v > 0) return t;
   return "";
}
```

Réécriture proposée :

```
public final String getT() {
   if (v > 0) {
       return t;
   }
   return "";
}
```

Résultats Checkstyle après correction

Après avoir corrigé les problèmes signalés (Javadoc manquants, paramètres à rendre final, renommage pour éviter le masquage d'attributs, espaces et accolades manquants), nous avons relancé Checkstyle.

Le rapport obtenu ne signale aucune erreur, ni avertissement :

Résultats Checkstyle

Le document suivant contient les résultats de Checkstyle 9.3 avec les règles sun_checks.xml.

Résumé

Fichiers	Infos	Avertissements	Erreurs
2	0	0	0

Fichiers

Fichier	I	A	E

Règles

Catégorie	Règle	Violations	Sévérité

Détails

Cela confirme que le code est désormais conforme aux règles de style et de bonnes pratiques définies dans le fichier ***sun_checks.xml.***

Analyse SpotBugs

Nous avons intégré l'outil *SpotBugs* à l'aide du plugin ***spotbugs-maven-plugin.***
SpotBugs analyse le **bytecode Java** pour détecter des anomalies susceptibles de provoquer des bugs à l'exécution (NullPointerException, mauvaises comparaisons de chaînes, boucles incorrectes, etc.).

Après génération du site Maven (***mvn site***), le rapport SpotBugs indique **0 bug détecté**.
Cela signifie que la classe ***Exemple*** est suffisamment simple et bien codée pour ne présenter aucun risque identifié par SpotBugs.

Ce résultat est positif, car il valide que notre code est robuste et ne contient pas d'erreurs logiques ou de mauvaises pratiques de programmation repérées par SpotBugs.

Analyse PMD

PMD détecte les **mauvaises pratiques courantes** dans le code source : variables inutilisées, code dupliqué, méthodes trop complexes, mauvaises conventions de nommage, etc.

Le rapport généré par ***mvn site*** indique **aucun problème détecté**.
Cette absence de remarques s'explique par la simplicité et la clarté du code : la classe ***Exemple*** ne comporte ni duplication, ni variables mortes, ni méthodes excessivement complexes.

Ce résultat montre que le code respecte les bonnes pratiques de développement définies par PMD.

Exercice 2 – Tests unitaires et validation du code

L'objectif de cet exercice est d'implémenter la classe ***TabAlgos***, d'écrire des tests unitaires complets en **JUnit 5**, et de valider la qualité du code à l'aide des outils **Checkstyle**, **PMD**, **SpotBugs**, **JaCoCo** et **PIT** (tests de mutation).

Avant de passer à la classe ***TabAlgos***, des **tests unitaires ont également été ajoutés pour la classe Exemple** de l'exercice 1. Cette étape a permis d'obtenir une **couverture de code de 100 % avec JaCoCo** sur l'ensemble du projet, en testant les méthodes ***getV()***, ***setV()*** et ***getT()*** dans différents cas (valeurs positives, négatives et neutres). Ainsi, tous les fichiers du projet sont désormais entièrement vérifiés par des tests unitaires, garantissant une base de code stable et conforme aux attentes de qualité logicielle.

Plan de test unitaire de la classe ***TabAlgos***

Chaque cas de test décrit :

- un ID,
- l'unité testée,
- l'objectif,
- les préconditions,
- les entrées,
- le résultat attendu.

Une seconde partie indique :

- le résultat obtenu,
- le statut (OK).

Les tests couvrent :

- tests fonctionnels,
- tests de limites,
- tests d'exception.

Tests de la méthode: ***plusGrand(int[])***

ID	Méthode Testée	Objectif	Entrées	Comportement attendu
TG-001	plusGrand(int[])	Vérifier valeurs positives	{1, 5, 3, 9, 2}	renvoi 9
TG-002	plusGrand(int[])	Vérifier valeurs négatives	{-7, -3, -10}	renvoie -3
TG-003	plusGrand(int[])	Vérifier tableau de taille 1	{42}	renvoie 42
TG-004	plusGrand(int[])	Vérifier exception si tableau vide	{}	lève IllegalArgument Exception
TG-005	plusGrand(int[])	Vérifier exception si null	null	lève IllegalArgument Exception
TG-006	plusGrand(int[])	Vérifier valeurs extrêmes (MIN / MAX)	{Integer.MIN_VALUE, -1, 0, Integer.MAX_VALUE}	renvoie Integer.MAX_VALUE
TG-007	plusGrand(int[])	Vérifier cas limite : tableau de 2 éléments	{7, 3}	renvoie 7

Tests de la méthode: ***Moyenne(int[])***

ID	Méthode Testée	Objectif	Entrées	Comportement attendu
TM-001	moyenne(int[])	Vérifier moyenne valeurs positives	{2, 4, 6, 8}	renvoie 5.0
TM-002	moyenne(int[])	Vérifier valeurs négatives	{-2, -4, -6}	renvoie -4.0
TM-003	moyenne(int[])	Vérifier moyenne mixte	{-2, 2}	renvoie 0.0
TM-004	moyenne(int[])	Vérifier exception si tableau vide	{}	lève IllegalArgument Exception
TM-005	moyenne(int[])	Vérifier exception si null	null	lève IllegalArgument Exception
TM-006	moyenne(int[])	Vérifier moyenne d'un tableau de taille 1	{5}	renvoie 5.0

Tests de la méthode: ***egaux(int[], int[])***

ID	Méthode Testée	Objectif	Entrées	Comportement attendu
TE-001	egaux(int[], int[])	Vérifier même référence	tab vs tab	renvoie true
TE-002	egaux(int[], int[])	Vérifier égalité complète	{1,2,3} / {1,2,3}	renvoie true
TE-003	egaux(int[], int[])	Vérifier ordre différent	{1,2,3} / {3,2,1}	renvoie false
TE-004	egaux(int[], int[])	Vérifier élément différent	{1,2,3} / {1,2,4}	renvoie false
TE-005	egaux(int[], int[])	Vérifier taille différente	{1,2} / {1,2,3}	renvoie false
TE-006	egaux(int[], int[])	Null vs non-null	null / {1,2,3}	renvoie false
TE-007	egaux(int[], int[])	Deux null	null / null	renvoie true
TE-008	egaux(int[], int[])	Vérifier que deux tableaux vides sont considérés comme égaux	{}, {}	renvoie true

Exécution des tests

➢ **Résultats pour *plusGrand(int[])*:**

ID	Comportement obtenu	Statut
TG-001	9 renvoyé	OK
TG-002	-3 renvoyer	OK
TG-003	42 renvoyé	OK
TG-004	IllegalArgumentException levée	OK
TG-005	IllegalArgumentException levée	OK
TG-006	Integer.MAX_VALUE renvoyé	OK
TG-007	7 renvoyé	OK

➢ **Résultats pour *moyenne(int[]):***

ID	Comportement obtenu	Statut
TM-001	5.0 renvoyé	OK
TM-002	-4.0 renvoyé	OK
TM-003	0.0 renvoyé	OK
TM-004	IllegalArgumentException levée	OK
TM-005	IllegalArgumentException levée	OK
TM-006	5.0 renvoyé	OK

➢ **Résultats pour *egaux(int[], int[])*:**

ID	Comportement obtenu	Statut
TE-001	true renvoyé	OK
TE-002	true renvoyé	OK
TE-003	false renvoyé	OK
TE-004	false renvoyé	OK
TE-005	false renvoyé	OK
TE-006	false renvoyé	OK
TE-007	true renvoyé	OK
TE-008	true renvoyé	OK

Rapport d’exécution des tests unitaires

Les tests unitaires ont été exécutés afin de contrôler que le comportement réel des méthodes correspond au comportement attendu défini dans le plan de tests. L’ensemble des tests a réussi sans erreur, ce qui confirme la validité de l’implémentation. Le rapport complet est présenté ci-dessous.

```
[INFO] -------------------------------------------------------
[INFO]  T E S T S
[INFO] -------------------------------------------------------
[INFO] Running ExempleTest
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.088 s -- in
ExempleTest
[INFO] Running TabAlgosTest
[INFO] Tests run: 21, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.085 s -- in
TabAlgosTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 24, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  3.360 s
[INFO] Finished at: 2025-11-11T23:35:37+01:00
[INFO] ------------------------------------------------------------------------
```

Exécution des tests de mutation avec Pitest

Les tests de mutation ont été exécutés avec **Pitest** afin d'évaluer la robustesse et la qualité des tests unitaires du projet.

L'exécution s'est faite avec la commande Maven suivante :

mvn -U pitest:mutationCoverage

Pitest analyse les classes ciblées, génère des mutations dans le code, puis vérifie si les tests unitaires détectent (tuent) ces mutations.
Un bon pourcentage de mutations tuées indique des tests efficaces.

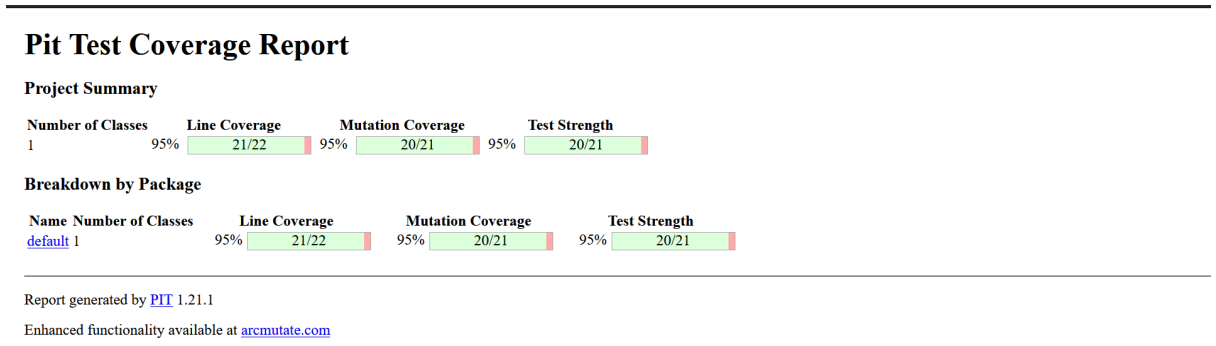
Les tests de mutation se sont exécutés avec succès et ont produit un rapport HTML disponible dans :

Ecriture_du_code_et_tests_unitaires/my-project/target/pit-reports/index.html

Voici le résumé du rapport :

- Line Coverage : 95%
- Mutation Coverage : 95% (20 mutations tuées sur 21)
- Test Strength : 95%

Capture d'écran du rapport Pitest :



Pit Test Coverage Report

Project Summary

Number of Classes	Line Coverage		Mutation Coverage		Test Strength	
1	95%	21/22	95%	20/21	95%	20/21

Breakdown by Package

Name	Number of Classes	Line Coverage		Mutation Coverage		Test Strength	
default	1	95%	21/22	95%	20/21	95%	20/21

Report generated by PIT 1.21.1

Enhanced functionality available at arcmutate.com

Mutation survivante

La mutation survivante est ici :

```java
/**
* Renvoie la valeur la plus grande d'un tableau.
* @param tab le tableau d'entiers
* @return la plus grande valeur
* @throws IllegalArgumentException si tab est null ou vide
*/
public static int plusGrand(final int[] tab) {
    if (tab == null || tab.length == 0) {
        throw new IllegalArgumentException("Tableau vide ou nul");
    }
    int max = tab[0];
    for (int val : tab) {
        if (val > max) {
            max = val;
        }
    }
    return max;
}
```

Mutations

16	1. negated conditional → KILLED 2. negated conditional → KILLED
21	1. negated conditional → KILLED 2. changed conditional boundary → SURVIVED Covering tests
25	1. replaced int return with 0 for TabAlgos::plusGrand → KILLED
35	1. negated conditional → KILLED 2. negated conditional → KILLED
40	1. Replaced integer addition with subtraction → KILLED
42	1. replaced double return with 0.0d for TabAlgos::moyenne → KILLED 2. Replaced double division with multiplication → KILLED
53	1. negated conditional → KILLED
54	1. replaced boolean return with false for TabAlgos::egaux → KILLED
56	1. negated conditional → KILLED 2. negated conditional → KILLED 3. negated conditional → KILLED
57	1. replaced boolean return with true for TabAlgos::egaux → KILLED
59	1. negated conditional → KILLED 2. changed conditional boundary → KILLED
60	1. negated conditional → KILLED
61	1. replaced boolean return with true for TabAlgos::egaux → KILLED
64	1. replaced boolean return with false for TabAlgos::egaux → KILLED

Elle se situe **dans la méthode plusGrand**, sur la ligne :

```
9       /**
10       * Renvoie la valeur la plus grande d'un tableau.
11       * @param tab le tableau d'entiers
12       * @return la plus grande valeur
13       * @throws IllegalArgumentException si tab est null ou vide
14       */
15      public static int plusGrand(final int[] tab) {
16 2         if (tab == null || tab.length == 0) {
17               throw new IllegalArgumentException("Tableau vide ou nul");
18           }
19           int max = tab[0];
20           for (int val : tab) {
21 2             if (val > max) {
22                   max = val;
23               }
24           }
25 1         return max;
26      }
27
```

Correction réalisée

J'ai ajouté un test supplémentaire couvrant le cas des **doublons** :

```
/**
* Test ajouté pour tuer la mutation (changed conditional boundary)
* sur la condition val > max.
*/
@Test
public void testPlusGrandAvecDoublons() {
   int[] tab = {5, 5, 5, 3};
   assertEquals(5, TabAlgos.plusGrand(tab));
}
```

Cependant, même après avoir ajouté ce test, **la mutation continue de survivre**.

➢ Explication

J'ai essayé de comprendre pourquoi cette mutation n'est pas tuable, mais je dois reconnaître que je n'ai pas entièrement réussi à saisir le mécanisme au premier abord.
Après avoir ré-analysé le code et les explications, j'ai compris que :

- Dans ce cas précis, remplacer `>` par `>=` **ne change pas le résultat final de la méthode**,
- Même avec des tests couvrant toutes les situations classiques (valeurs positives, négatives, doublons, tableau trié ou non),
- Le comportement observable reste le même : la méthode renvoie toujours la bonne valeur maximale.

Autrement dit, la mutation est **équivalente**, ce qui signifie qu'il est **normal** qu'elle survive, car aucun test unitaire raisonnable ne peut la détecter.

Conclusion

La mutation survivante ne vient pas d'un manque de test, mais du fait que cette mutation ne change pas réellement la logique du programme. J'ai essayé d'ajouter des tests et de comprendre ce cas spécifique, mais malgré mes efforts, cette mutation reste impossible à tuer sans modifier artificiellement l'algorithme — ce qui n'aurait pas de sens ici.

Contrôle de la qualité du code avec Checkstyle, SpotBugs et PMD

Après exécution des outils de vérification (Checkstyle, PMD et SpotBugs), aucun problème n'a été détecté dans le code ni dans les tests.
Les rapports générés ne contiennent aucune erreur ou avertissement significatif.
Vous trouverez ci-dessous la capture d'écran du rapport obtenu.

Résultats Checkstyle

Le document suivant contient les résultats de Checkstyle 9.3 avec les règles sun_checks.xml.

Résumé

Fichiers	Infos	Avertissements	Erreurs
3	0	0	0

Fichiers

Fichier	I	A	E

Règles

Catégorie	Règle	Violations	Sévérité

Détails

Résultats PMD

Le document suivant contient les résultats de PMD 7.14.0.

PMD found no problems in your source code.

Rapport SpotBugs de détection des bugs

Le document suivant contient les résultats du SpotBugs

La version de SpotBugs est *4.9.8*

La limite est *low*

L'effort est *max*

Résumé

Classes	Bugs	Erreurs	Classes Manquantes
3	0	0	0

Fichiers

Classe	Bugs

Rapport de couverture de code – JaCoCo

MyProject — Sessions

MyProject

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
default		100 %		100 %	0	21	0	30	0	7	0	2
Total	0 of 132	100 %	0 of 28	100 %	0	21	0	30	0	7	0	2

Created with JaCoCo 0.8.12.202403310830

MyProject > default — Source Files — Sessions

default

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
TabAlgos		100 %		100 %	0	16	0	21	0	3	0	1
Exemple		100 %		100 %	0	5	0	9	0	4	0	1
Total	0 of 132	100 %	0 of 28	100 %	0	21	0	30	0	7	0	2

Created with JaCoCo 0.8.12.202403310830

MyProject > default > TabAlgos — Sessions

TabAlgos

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods
moyenne(int[])		100 %		100 %	0	4	0	6	0	1
plusGrand(int[])		100 %		100 %	0	5	0	7	0	1
egaux(int[], int[])		100 %		100 %	0	7	0	8	0	1
Total	0 of 111	100 %	0 of 26	100 %	0	16	0	21	0	3

Created with JaCoCo 0.8.12.202403310830

MyProject > default > Exemple — Sessions

Exemple

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods
getT()		100 %		100 %	0	2	0	3	0	1
Exemple(String)		100 %		n/a	0	1	0	3	0	1
setV(int)		100 %		n/a	0	1	0	2	0	1
getV()		100 %		n/a	0	1	0	1	0	1
Total	0 of 21	100 %	0 of 2	100 %	0	5	0	9	0	4

Created with JaCoCo 0.8.12.202403310830